OPTI-CROP PROJECT DELIVERABLE

Code Layout, Readability, and Reusability

1. Coding Standards and Best Practices

The Opti-Crop codebase implements clean code guidelines, focusing on:

- **PEP 8 Compliance:** Python files follow clear indentation, descriptive function naming, and appropriate space formatting.
- **Resource Management:** Database connections are handled using Python's context managers or explicit connection-closing patterns to prevent thread locks.
- **Error Handling:** Model loading (`model.pkl`, `crop_ranges.pkl`) is wrapped inside try-except blocks with robust fallback parameters to prevent server crashes on startup.

2. Directory Structure Modularization

The codebase has been refactored to cleanly segregate concerns:

- **`database.py`:** Dedicated module for database initialization, table creation, seed data inputs, and database connection pooling.
- **`train_model.py`:** Separate script for analytics, visualization asset generation, data cleaning, and ML training.
- **`app.py`:** Core web controller file hosting Flask route endpoints, prediction handlers, and suitability calculators.
- **`static/` & `templates/`:** Isolated layout assets (stylesheets, plots, images) and HTML view templates.

3. Code Reusability Highlights

- **Context Injection:** Shared variables (like the list of crops from Pickle ranges) are injected globally into Flask templates using `@app.context_processor`, eliminating redundant database queries.
- **Shared DB Connection:** `get_db_connection()` is central, returning row-factory SQLite connections that allow accessing attributes by column name.